Protocol Audit Report

Date: July 12, 2026

Prepared by: Audit Team

Lead Auditors: Vraj Parikh

Table of Contents

- Protocol Audit Report
- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
    - [H-1] Storing the password on-chain makes it visible to anyone and no longer private (Root Cause + Impact)
    - [H-2] `PasswordStore::setPassword` has no access controls, mneaning a non-owner can change the password
  - Informational
    - [I-1] The `PasswordStore::getPassword` natspec indicates a parameter that does not exist, causing netspec to be incorrect

Protocol Summary

PasswordStore is a protocol to store and retrieve user's password. The protocol is designed to be used by a single user and only the woner should be able to set anc access the password.

Disclaimer

The Audit team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

** The findings described in the section are from the following commit hash:

```
f356e2c88ae5cd6036daa5bcdd893bd32c7d25ab
```

Scope

```
./src/
#-- PasswordStore.sol
```

Roles

- Owner: The user who can set the password and read the password.
- Outsides: No one else should be able to set or read the password.

Executive Summary

Notes about how the audit went, how many hours spent, etc.

Issues found

Severity	Number of issues found
High	2
Medium	0
Low	0
Info	1
Gas Optimizations	0
Total	3

Findings

High

[H-1] Storing the password on-chain makes it visible to anyone and no longer private (Root Cause + Impact)

Description: All data stored on chain is visible to anyone and can be read directly from blockchain. The `PasswordStore::s_password` variable is intended to be a private variable and only accessed through the `PasswordStore::getPassword` function which is intended to be only called by the owner of the contract

We show one such method of reading data off chain below:

Impact:

Proof of Concept:

The below test case shows that anyone can read the password directly from the blockchain

1. Create a locally running chain

```
make anvil
```

2. Deploy the contract to the chain

```
make deploy
```

3. Run the storage tool

We use `1` because that's the storage slot of `s_password` in the contract.

```
cast storage <ADDRESS_HERE> 1 --rpc-url http://127.0.0.1:8545
```

You'll get an output that looks like this:

`0x6d7950617373776f726400000000000000000000000000000000000000000014`

You can then parse that hex to a string with:

```
cast parse-bytes32-string 0x6d7950617373776f726400000000000000000000000000000000000000000014
```

And get an output of:

```
myPassword
```

Recommended Mitigation: Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain, and then store the encrypted password on-chain. This would require the user to remember another password off-chain to decrypt the password. However, you'd also likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with the password that decrypts your password.

[H-2] `PasswordStore::setPassword` has no access controls, mneaning a non-owner can change the password

Description: The `PasswordStore::setPassword` function is set to be external function however the natspec of function and overall purpose of smart contract is that `The function allows only owner to change the password`

```
    function setPassword(string memory newPassword) external {
@>      //@audit There is no access control
        s_password = newPassword;
        emit SetNewPassword();
    }
```

Impact: Anyone can change the password of the contract, breaking the smart contract intended functionality

Proof of Concept:

Add the following to the `PasswordStore.t.sol` test suite.

```
function test_anyone_can_set_password(address randomAddress) public {
    vm.prank(randomAddress);
    string memory expectedPassword = "myNewPassword";
    passwordStore.setPassword(expectedPassword);

    vm.prank(owner);
    string memory actualPassword = passwordStore.getPassword();
    assertEq(actualPassword, expectedPassword);
}
```

Recommended Mitigation: Add an access control condition in `setPassword function`

```
if(msg.sender != s_owner){
    revert PasswordStore_NotOwner();
}
```

Informational

[I-1] The `PasswordStore::getPassword` natspec indicates a parameter that does not exist, causing netspec to be incorrect

Description:

```
    /*
     * @notice This allows only the owner to retrieve the password.
@>   * @param newPassword The new password to set.
     */
    function getPassword() external view returns (string memory)
```

The `PasswordStore::getPassword` function signature is `getPassword()` while natspec says it should be `getPassword(string)`

Impact: The natspec is incorrect

Recommended Mitigation: Remove the incorrect natspec line

```
-     * @param newPassword The new password to set.
```